

SalesOS Production-Ready Claude Code Prompt Package and Execution Report

Executive summary

The strongest way to operationalize this entire conversation is to convert it into a **repository-native operating package** for Claude Code, not a single giant one-shot prompt. Anthropic's own Claude Code guidance is explicit that project behavior is best stabilized through a concise `CLAUDE.md`, permission controls, subagents, checkpoints, non-interactive runs, and a fresh-context adversarial review step before considering work "done." Anthropic also emphasizes that context is the central constraint, which is why a layered package—Constitution, Product Bible, Architecture, Execution Book, and task-specific prompts—will perform better than a single monolithic prompt that bloats context and degrades over time. ¹

The architecture that best fits the decisions made in this chat is a **decision-centric, multi-agent operating system**: a Supreme Orchestrator on top of specialized departmental agents, an Organizational DNA Engine to enforce S.O.S. style and standards, a Company Memory/Knowledge Graph to preserve reusable institutional knowledge, and a strict Agent Communication Protocol so the system remains auditable, reversible, and governable. This architecture should be paired with durable workflow execution, structured telemetry, and a relational core. Temporal's durable execution model is well-suited to long-running orchestration with retries and failure handling, PostgreSQL provides row-level security and `jsonb`, pgvector provides in-database vector search with HNSW indexing, and OpenTelemetry gives vendor-neutral observability across traces, metrics, and logs. ²

Production readiness also hinges on treating integrations as first-class engineering workstreams. Meta requires explicit permissions and App Review for Instagram and WhatsApp use cases; Google OAuth requires secure redirect URIs and token handling; Canva Connect uses OAuth 2.0 Authorization Code + PKCE and per-user scopes; OpenAI's Responses API is its forward path for tool-using agents; Anthropic recommends prompt caching and tool descriptions; Gemini rate limits and key behavior vary by tier and should be read from AI Studio rather than hardcoded. These facts justify dedicated integration playbooks, approval gates, and "external action" tasks inside the Execution Book. ³

The recommended implementation path is therefore: **build the book, wire Claude Code to the book, and then drive execution through task cards**. The report below consolidates the chat into that package.

Consolidated decisions and product constitution

SalesOS, as defined by the conversation, is not a CRM, not an LMS, not a chatbot, and not a generic AI wrapper. It is a **digital operating company** for S.O.S., built to centralize knowledge, decisions, student operations, content production, research, competitor intelligence, analytics, automation, and communications under one system. The central philosophical choice is that the product is **decision-centric**

rather than task-centric: the system should surface what should happen and why, then either execute, queue for approval, or recommend only, depending on governance level.

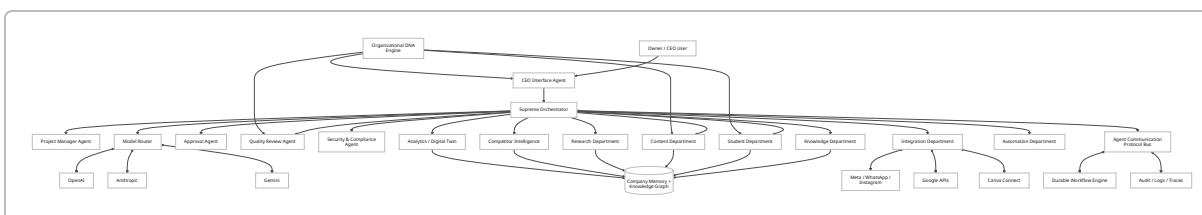
The following table consolidates the non-negotiables expressed across the chat into a usable constitution for Claude Code.

Area	Consolidated decision
Product identity	SalesOS is a multi-agent AI Operating System for S.O.S.
Architectural stance	Build as a company of specialized agents, not one omnipotent chat agent
Control model	AI suggests and prepares; humans retain final authority over material business actions
Business rule	Every feature must save time, improve quality, or improve efficiency; it must also be automatable and scalable
Information model	One Source of Truth; no duplicate canonical records
Knowledge model	All knowledge passes through capture → validation → categorization → relationship mapping → approval → production → versioning
Safety	AI never guesses when confidence is low; it asks, searches, or says it does not know
Reversibility	Every meaningful action must be reversible or compensatable
Auditability	Everything is logged: prompts, tool calls, outputs, approvals, rejections, and external API actions
Memory design	Each agent has isolated working memory; promotion to central memory is controlled
Governance	Approval levels determine whether the system can execute, notify, ask, or only recommend
Integrations	Only official / compliant APIs and approved flows; no scraping that violates platform terms
Product scope	Build platform core for S.O.S. first, but architect for multi-tenant future packages
Explainability	The system must explain why it recommended or executed an action
Execution model	Claude Code acts as CTO + Technical Product Manager + senior engineer, but works task-by-task against a formal Execution Book
Delivery discipline	No coding without architecture, acceptance criteria, tests, docs, and rollback considerations
Future-proofing	Vendor-agnostic model router; no hard dependency on a single LLM provider

The Master Blueprint implied by the conversation resolves into the following modules.

Module	Purpose
Executive Office	CEO interaction, orchestration, priorities, approvals
Identity and Tenant Management	Organizations, users, roles, memberships, tenancy
Agent Runtime	Agent registry, routing, task execution, subagent delegation
Agent Communication Protocol	Internal message schema, lifecycle, traceability
Organizational DNA Engine	Brand rules, content guardrails, S.O.S. language and standards
Company Memory and Knowledge Graph	Documents, chunks, relationships, semantic retrieval, versioning
Student OS	Student profiles, risk, progress, assignments, interventions
Education OS	Courses, lessons, recordings, transcripts, summaries
Content Engine	Ideas, drafts, briefs, assets, repurposing, publishing plans
Research and Competitor Intelligence	Market, psychology, industry, trend, competitor observation
Communications Hub	WhatsApp, Instagram, Gmail, internal notifications
Automation and Approval Engine	Workflows, queues, triggers, SLA timers, approval routing
Analytics and Digital Twin	KPI dashboards, forecasts, simulations, what-if analysis
Integration Platform	OAuth, tokens, webhooks, provider adapters, retries
Security and Compliance	RBAC, secret handling, audit, data retention, privacy
Frontend Workspaces	CEO console, ops dashboards, review queues, authoring views
Developer Platform	Monorepo, standards, test harnesses, seed data, CI/CD
Operations	Backups, monitoring, incident response, cost controls

The recommended structural view is below.



Master blueprint and agent system

The conversation repeatedly converged on the idea that SalesOS should operate like a company, not like a chat session. That aligns well with Claude Code’s support for specialized subagents, fresh-context reviews, and task isolation. Anthropic specifically recommends subagents for investigation and review because they preserve main-context quality and keep large searches from polluting the principal workstream. 4

Agent catalog

The catalog below defines a **core set of twenty-four agents**. This is intentionally more than the “12 departments” discussed earlier, because the departments become operationally safer when split into narrow, typed responsibilities.

Approval legend:

- L0 autonomous within policy
- L1 autonomous but notify
- L2 require manager approval
- L3 require CEO approval
- L4 recommend only

Confidence legend:

- High : may proceed if approval level allows
- Medium : must validate or ask for more context
- Low : cannot act; must escalate

Agent	Responsibility	Primary inputs	Primary outputs	Memory scope	Confidence threshold	Default approval
Supreme Orchestrator	Decompose intent, assign agents, maintain workflow state	User objective, system state, policies	Task graph, routing plan, completion state	Workflow-only plus references	0.85 to proceed with routing	L2
CEO Interface Agent	Conversational front door for owner	User chat, recent priorities	Natural-language brief, decision requests	Episodic session memory	0.80	L2-L4
Project Manager Agent	Convert goals into epics/features/tasks and status reports	Blueprint, backlog, current repository state	Sprint plans, task cards, blocker reports	Persistent project memory	0.90	L1

Agent	Responsibility	Primary inputs	Primary outputs	Memory scope	Confidence threshold	Default approval
Model Router Agent	Choose Claude/ OpenAI/Gemini/ model class per subtask	Task type, budget, latency, policy	Model selection, failover record	Stateless + usage telemetry	0.90	L0
Brand DNA Agent	Enforce S.O.S. tone, prohibited claims, style rules	Draft content, lesson summaries, messages	Pass/fail, edit suggestions, violations	Long-lived brand memory	0.95	L0-L2
Knowledge Intake Agent	Capture raw knowledge from files, calls, URLs, uploads	PDFs, docs, transcripts, URLs	Candidate knowledge items	Source-scoped episodic memory	0.80	L1
Knowledge Curator Agent	Validate, classify, normalize, and version knowledge	Candidate items, metadata, sources	Approved knowledge items, tags	Department memory + KG refs	0.90	L2
Knowledge Graph Agent	Maintain relationships among entities and concepts	Approved knowledge items, BOM entities	Graph edges, retrieval plans	Central graph memory	0.92	L1
Student Intelligence Agent	Maintain student profiles, progress, risk, style	Assignments, lesson data, communications	Student scorecards, risk signals	Per-student departmental memory	0.85	L1-L2
Coach Support Agent	Recommend interventions to coaches	Student signals, lesson summaries	Coach action suggestions	Per-student plus coach notes	0.80	L2
Lesson Agent	Process lesson events	Recording, attendee list, transcript	Summary, action items, knowledge proposals	Per-lesson episodic memory	0.88	L1

Agent	Responsibility	Primary inputs	Primary outputs	Memory scope	Confidence threshold	Default approval
Assignment Review Agent	Review, score, and suggest revisions	Submission, rubric, lesson context	Feedback, score, revision requests	Per-assignment memory	0.85	L2
Research Agent	Gather market, sales, psychology, and AI research	Research brief, domains, constraints	Research packet, citations, insights	Research-session memory	0.90 if cited	L1-L2
Competitor Intelligence Agent	Track public competitor content and patterns	Approved public sources, handles, URLs	Observations, trends, opportunity flags	Competitor dossier memory	0.88	L1-L2
Opportunity Agent	Turn observations into concrete business opportunities	Insights, competitor data, analytics	Opportunity memo, expected impact	Cross-functional working memory	0.82	L2
Content Strategy Agent	Decide what content should exist and why	Opportunities, brand rules, KPIs	Content brief, content plan	Campaign memory	0.88	L2
Copy Agent	Generate copy and structured content assets	Approved content brief, DNA rules	Draft scripts, captions, carousels	Draft-local memory	0.90	L2
Design Brief Agent	Convert content into Canva-ready briefs	Content asset, brand kit, format rules	Design brief, export specs	Asset-local memory	0.90	L2
Publishing Readiness Agent	Check content before publish	Final asset, platform rules, CTA links	Publish approval packet	Asset-local memory	0.95	L3

Agent	Responsibility	Primary inputs	Primary outputs	Memory scope	Confidence threshold	Default approval
Communications Agent	Draft or send student/customer communications	Message objective, recipient context	Message draft, send request, thread note	Thread memory	0.88	L2-L3 for sends
Analytics Agent	Produce KPI and operational analytics	Event logs, business metrics, entity state	Dashboards, anomalies, trends	Central analytical memory	0.92	L1
Digital Twin Agent	Run simulations and scenario analysis	Business metrics, constraints, assumptions	Forecasts, what-if memos	Simulation session memory	0.80	L4 by default
Automation Agent	Execute workflow logic and schedules	Triggers, workflow definitions, approvals	Workflow runs, job states	Workflow memory	0.95	L0-L2 depending action
Integration Agent	Manage provider adapters, OAuth, webhooks, tokens	Provider configs, API requests, secrets refs	Connection status, webhook events, sync jobs	Connector-scoped memory	0.95	L1-L2
Security and Compliance Agent	Check RBAC, secrets, retention, and audit coverage	Planned action, policy, scopes	Permit/deny, remediation notes	Policy memory only	0.98	L3-L4 on critical actions
Quality Review Agent	Fresh-context review of outputs and diffs	Diff, artifact, acceptance criteria	Findings, severity, accept/reject	Fresh temporary context	0.92	L1
Learning Promotion Agent	Decide what local memory can graduate to central knowledge	Local agent results, usage stats, approvals	Promotion proposal, version note	Learning pipeline memory	0.90	L2

Agent Communication Protocol

The Agent Communication Protocol is the difference between “many prompts” and a system. It should be implemented as a strict internal contract. Anthropic’s guidance on tool descriptions, context engineering, and explicit system structure strongly supports this approach, because agentic quality improves when instructions, tools, and context are well-curated rather than improvised turn by turn. ⁵

The ACP should use the following canonical message schema:

```
{
  "message_id": "uuid",
  "workflow_id": "uuid",
  "task_id": "TASK-KG-004",
  "parent_task_id": "TASK-KG-001",
  "correlation_id": "uuid",
  "causation_id": "uuid",
  "tenant_id": "org_sos",
  "source_agent": "supreme_orchestrator",
  "target_agent": "knowledge_curator",
  "requested_by": "ceo_interface|system|workflow",
  "priority": "critical|high|medium|low|background",
  "approval_level": "L0|L1|L2|L3|L4",
  "required_confidence": 0.9,
  "objective": "Promote validated lesson insight into production knowledge",
  "constraints": {
    "deadline": "2026-07-10T10:00:00Z",
    "budget_tokens": 12000,
    "max_retries": 2,
    "must_cite_sources": true,
    "provider_restrictions": ["official_api_only"]
  },
  "context_refs": [
    "kg:item:abc123",
    "lesson:42",
    "student:998"
  ],
  "inputs": {
    "payload_ref": "s3://...",
    "inline_summary": "..."
  },
  "status": "created|queued|in_progress|blocked|awaiting_review|
awaiting_approval|completed|failed|rolled_back",
  "output": null,
  "evidence": [],
  "explainability": {
    "why_this_agent": "",
    "why_this_model": ""
  }
}
```

```

    "important_assumptions": []
  },
  "audit": {
    "created_at": "",
    "started_at": "",
    "completed_at": "",
    "cost_estimate_usd": 0.0,
    "provider": "anthropic",
    "model": "claude-sonnet",
    "token_usage": {
      "input": 0,
      "output": 0,
      "cached_input": 0
    }
  },
  "error": null,
  "next_action": null
}

```

The task lifecycle should be:

```

created → triaged → planned → queued → in_progress → self_validated → qa_review
→ awaiting_approval → completed

```

Alternate branches:

```

in_progress → blocked → escalated
in_progress → failed → retried → in_progress
completed → rolled_back when a compensating action is required

```

The priority model should be standardized:

Priority	Use case	SLA target
Critical	Webhook failures, auth failures, production outages, blocked approvals on time-sensitive ops	Minutes
High	Student-risk alerts, expiring tokens, launch blockers, broken publish path	Same day
Medium	Feature implementation, knowledge curation, content generation	1-3 days
Low	Back-office cleanup, refactors, low-risk enrichment	This sprint
Background	AI intelligence scans, model evaluations, passive competitor monitoring	Opportunistic

The confidence policy should be hard-coded into orchestration:

Confidence band	Meaning	Allowed behavior
0.90–1.00	Strong support, sufficient evidence	Execute if approval level allows
0.75–0.89	Plausible but not decisive	Route through QA or ask for confirmation
0.60–0.74	Weak or incomplete	Do not execute; ask for more data or research
< 0.60	Unsafe / speculative	Return insufficiency and escalate

The traceability and observability requirements should be non-negotiable. Every ACP message should emit correlated traces, structured logs, and audit events; OpenTelemetry is specifically designed to correlate traces, metrics, and logs in a vendor-neutral way. ⁶

Explainability should always include five fields:

1. Why this action was proposed
2. Which sources and internal records were used
3. Which rules or DNA policies affected the result
4. Which alternatives were considered or rejected
5. Whether a human approval gate was applied

Error recovery should distinguish between **transient**, **deterministic**, and **policy** failures:

Failure type	Example	Response
Transient infrastructure	429, 502, timeout, DB lock	Retry with exponential backoff and jitter; obey <code>retry-after</code> if present
Deterministic validation	Acceptance test failed, schema mismatch, failing contract test	Do not blind-retry; revise implementation or escalate
Policy failure	Missing approval, insufficient permission, prohibited action	Hard stop; create approval or remediation task
Context failure	Missing required facts, ambiguous entity mapping	Ask for missing context or trigger research subtask
Provider outage	Meta or Google service disruption	Open circuit breaker, queue retry, notify ops

Data model and governance

The conversation defined a Business Object Model in which everything important is an object, and the system should model the business rather than incidental implementation details. That points to a relational core with a graph-friendly overlay and vector retrieval. PostgreSQL is appropriate because it gives

structured relational modeling, `jsonb` for flexible metadata, and row-level security for tenant and role isolation. pgvector complements it with in-database similarity search. ⁷

Business object model

The BOM below is the recommended canonical set for SalesOS v1.

Object family	Canonical objects
Core business	Organization, User, Role, Membership, Student, CoachProfile, Course, Cohort, Lesson, Assignment, Submission, Feedback
Knowledge	KnowledgeItem, KnowledgeChunk, KnowledgeVersion, KnowledgeEdge, SourceDocument, Transcript, Recording
Decisioning	Decision, Insight, Recommendation, Opportunity, Approval
Content	ContentAsset, ContentVersion, Campaign, BrandRule, DesignBrief, PublishJob
Intelligence	ResearchBrief, ResearchArtifact, Competitor, CompetitorObservation, Trend, SimulationRun
Automation	Workflow, WorkflowVersion, WorkflowRun, TaskRun, Notification
Integrations	IntegrationConnection, OAuthCredential, WebhookEndpoint, WebhookEvent, ProviderJob
Systems and controls	Agent, AgentMemory, AuditEvent, TraceLink, ModelInvocation, CostLedger

ERD table

Entity	Key attributes	Relationships
organizations	id, name, slug, package_type, status	1:N memberships, integrations, students, knowledge_items
users	id, email, display_name, status	1:N memberships, approvals, comments, sessions
roles	id, code, description	N:M users via memberships
memberships	id, org_id, user_id, role_id, permissions_json	Belongs to organization, user, role
students	id, org_id, primary_coach_user_id, stage, motivation_score, dropout_risk, retention_score, learning_style	N:1 organization, N:1 coach, 1:N submissions, lesson_attendance
coach_profiles	user_id, specialties, max_capacity, active_flag	1:1 users, 1:N students

Entity	Key attributes	Relationships
courses	id, org_id, title, version, status	1:N lessons, cohorts
cohorts	id, org_id, course_id, name, start_at, end_at	N:1 course, N:M students
lessons	id, org_id, course_id, scheduled_at, teacher_user_id, recording_id, transcript_id	N:1 course, 1:1 transcript, 1:N action_items
recordings	id, org_id, provider, storage_uri, duration_sec	1:1 lesson
transcripts	id, org_id, lesson_id, text, stt_provider, quality_score	1:1 lesson, 1:N knowledge_items
assignments	id, org_id, course_id, lesson_id, rubric_json, due_at	1:N submissions
submissions	id, assignment_id, student_id, submitted_at, score, revision_required	N:1 assignment, N:1 student, 1:N feedback
feedback	id, submission_id, author_user_id, ai_generated, content, version	N:1 submission
decisions	id, org_id, type, objective, status, created_by, approval_level	1:N insights, recommendations, approvals
insights	id, org_id, decision_id, type, confidence, summary	N:1 decision
recommendations	id, org_id, decision_id, action_type, confidence, rationale	N:1 decision
approvals	id, org_id, subject_type, subject_id, requested_by, approver_user_id, status	Polymorphic to decisions, publish jobs, workflows
knowledge_items	id, org_id, type, title, status, source_type, source_ref, current_version_id	1:N versions, 1:N chunks, N:M via edges
knowledge_versions	id, knowledge_item_id, version_no, content_hash, approved_by, approved_at	N:1 knowledge_item
knowledge_chunks	id, knowledge_item_id, seq, content, embedding, token_count, metadata_json	N:1 knowledge_item
knowledge_edges	id, org_id, from_item_id, to_item_id, relation_type, weight	N:1 item to item graph
brand_rules	id, org_id, rule_type, rule_text, severity, active	Used by DNA engine

Entity	Key attributes	Relationships
content_assets	id, org_id, type, title, status, target_channel, source_decision_id	1:N content_versions, 1:1 design_brief
content_versions	id, content_asset_id, version_no, body_json, qa_status	N:1 content_asset
design_briefs	id, content_asset_id, canva_template_id, format, constraints_json	1:1 content_asset
campaigns	id, org_id, objective, target_audience, status	1:N content_assets
competitors	id, org_id, name, handle, channels_json, tracking_status	1:N observations
competitor_observations	id, competitor_id, observed_at, url, content_type, summary, signals_json	N:1 competitor
research_briefs	id, org_id, question, scope_json, status	1:N research_artifacts
research_artifacts	id, research_brief_id, source_url, citation, relevance_score, summary	N:1 research_brief
workflows	id, org_id, code, name, trigger_type, approval_policy	1:N workflow_versions, runs
workflow_versions	id, workflow_id, semver, definition_json, status	N:1 workflow
workflow_runs	id, workflow_id, version_id, status, started_at, ended_at	1:N task_runs
task_runs	id, workflow_run_id, task_id, agent_code, priority, status, attempt_no	N:1 workflow_run
integration_connections	id, org_id, provider, scopes_json, status, connected_by	1:N oauth_credentials, webhook_endpoints, provider_jobs
oauth_credentials	id, integration_connection_id, token_ref, refresh_ref, expires_at, subject	N:1 integration_connection
webhook_endpoints	id, integration_connection_id, provider, url, verification_status	1:N webhook_events
webhook_events	id, webhook_endpoint_id, provider_event_id, received_at, processed_at, status, payload_json	N:1 endpoint
agents	id, code, department, model_policy_json, active	1:N model_invocations, agent_memories

Entity	Key attributes	Relationships
agent_memories	id, agent_id, scope_type, scope_ref, summary, promoted_flag	N:1 agent
model_invocations	id, agent_id, provider, model, prompt_hash, tokens_in, tokens_out, cost_usd	N:1 agent
audit_events	id, org_id, actor_type, actor_id, action, subject_type, subject_id, trace_id, payload_json	Global audit ledger
reports	id, org_id, type, generated_at, period_start, period_end, body_json	Derived analytics
kpi_snapshots	id, org_id, metric_code, ts, value, dimensions_json	Analytics base table

Governance model

The approval model should be formalized exactly as the conversation described it:

Level	Meaning	Examples
L0	AI can execute autonomously within narrow safe bounds	Tagging, deduping, internal draft formatting
L1	AI can execute and notify	Knowledge classification, low-risk dashboard refresh
L2	AI prepares and requests approval	Assignment feedback release, workflow activation
L3	CEO approval required	Sending external campaigns, high-impact student interventions, policy changes
L4	Recommendation only	Strategic changes, pricing ideas, major curriculum redesign

The knowledge promotion pipeline should be:

```
capture → validate → categorize → relationship map → review → approval →
production knowledge → version → archive
```

This should never be bypassed. Anthropic’s emphasis on context engineering and prompt caching strongly supports keeping the **stable** parts of the system—constitution, brand DNA, tool definitions, recurring process rules—separate from ephemeral, high-churn context so that repeated runs are cheaper and more stable. ⁸

Rollback and versioning rules should be universal:

Artifact type	Versioning rule	Rollback rule
Knowledge item	Immutable versions; one current pointer	Repoint current version to prior approved version
Workflow	Semver + activation window	Re-activate previous stable workflow version
Content	Draft and approved versions	Revert publish plan or republish prior approved asset
Prompt package	Git-tracked files with changelog	Git revert or restore checkpoint
OAuth / integrations	Versioned config records, never overwrite silently	Roll back config version; preserve event log
Schema	Forward-only migrations plus reversible down plan for dev	DBA-reviewed rollback or compensating migration

Claude Code prompt package and execution book

Anthropic's official recommendation is to use a concise, project-specific `CLAUDE.md`, keep it in git, use subagents for focused tasks, use auto mode carefully, and insist on verifiable checks that Claude can run before it declares work done. The package below turns that guidance into a SalesOS-specific operating harness. ⁹

Root prompt package for Claude Code

Repository-level `CLAUDE.md`

```
# SalesOS Claude Code Operating Contract

You are Claude Code acting as:
1. CTO
2. Technical Product Manager
3. Lead Architect
4. Senior Full-Stack Engineer
5. QA / Documentation owner

Your job is not to "hack together code".
Your job is to build SalesOS according to the Product Bible and Execution Book.

## Mandatory reading order before any task
1. docs/00-constitution/constitution.md
2. docs/01-vision/company-model.md
3. docs/02-architecture/master-blueprint.md
4. docs/03-data/bom-and-erd.md
5. docs/04-governance/ai-governance.md
```

6. docs/20-execution/current-sprint.md
7. The specific task card you were assigned

Non-negotiable rules

- Never break existing behavior without documenting it.
- Never start implementation before writing an Implementation Brief.
- Every non-trivial change must include tests.
- Every integration must include retries, timeout handling, logging, and docs.
- Every API route must emit structured logs and trace ids.
- Every AI action must be auditable and explainable.
- Every destructive operation must support rollback or compensation.
- Do not invent provider scopes or endpoints. Use official docs only.
- If requirements conflict, stop and create an ADR.
- If confidence is below 0.75, do not proceed silently.

Delivery contract for each task

Before coding:

- Restate the task.
- List dependencies and assumptions.
- Identify files to create/update.
- Identify DB/API/integration impact.
- Identify risks and rollback path.

During coding:

- Follow the repo standards.
- Reuse existing patterns before adding new abstractions.
- Keep functions small and typed.
- Prefer explicit contracts over hidden magic.

Before marking done:

- Run lint, typecheck, tests, and task-specific verification.
- Update docs and changelog if behavior changed.
- Produce the PM Mode report exactly.

PM Mode report format

- Completed
- Files changed
- DB changes
- APIs changed
- Tests added/updated
- Docs updated
- Risks / caveats
- Blockers
- External actions required
- Recommended next task

Permission policy

- Safe read-only investigation: proceed.

- Local file edits within task scope: proceed.
- Secrets, production infra, provider dashboards, destructive commands: require explicit approval.
- If using auto mode, still stop on high-risk actions.

Repository conventions

- Package manager: pnpm
- Workspace orchestration: turbo
- Use project commands from package.json
- Prefer targeted tests over whole-suite runs when possible
- Commit checklist: lint + typecheck + relevant tests must pass

Task invocation prompt template

You are executing TASK_ID={{TASK_ID}} for SalesOS.

Read first:

- docs/00-constitution/constitution.md
- docs/02-architecture/master-blueprint.md
- docs/03-data/bom-and-erd.md
- docs/20-execution/task-cards/{{TASK_ID}}.md

Your responsibilities:

1. Produce an Implementation Brief before touching code.
2. Validate dependencies and identify blockers.
3. Implement only the scope in this task.
4. Reuse existing patterns and shared packages.
5. Add or update tests.
6. Update task documentation and status.
7. Return the PM Mode report.

If required information is missing or contradictory:

- STOP implementation
- create docs/adr/ADR-{{date}}-{{slug}}.md
- explain the decision needed
- recommend the best option with tradeoffs

Definition of done:

- Acceptance criteria satisfied
- Tests pass
- No undocumented breaking changes
- Rollback path documented
- PM Mode report returned

Subagent template

```
---
name: security-reviewer
description: Reviews diffs for security, RBAC, data handling, and secrets leaks
tools: Read, Grep, Glob, Bash
model: opus
---
```

You are the SalesOS Security Reviewer.

Review only against:

- task acceptance criteria
- OWASP-style risks
- RBAC/tenant-isolation violations
- secret or token exposure
- audit/logging omissions
- unsafe retries or missing backoff

Return:

- findings with severity
- exact file references
- required fixes
- pass/fail

PM Mode checklist after every task

Item	Required output
Completed	What was actually implemented
Files changed	Exact files added/modified/deleted
DB changes	Migration names, schema impact, rollback note
APIs changed	Routes, contracts, providers, webhooks
Tests	Unit/integration/e2e coverage added or updated
Documentation	Docs pages and task card updated
Risks	Known limitations, assumptions, debt
Blockers	Remaining blockers, if any
External actions required	Accounts, OAuth, App Review, DNS, secrets, approvals
Recommended next task	Single next-best task with rationale

Execution book summary by module

All estimates below assume **no specific constraint** and one primary Claude Code workstream with occasional review subagents.

Module	Epics and features	Representative tasks	Est. effort	Dependencies	External actions
Foundation	Repo scaffold, monorepo, coding standards, local dev, CI	Create pnpm/turbo workspace; root apps/packages; Docker compose; GitHub Actions; base env loader; CLAUDE.md	4–6 days	None	GitHub org/repo, package registry, cloud account
Identity and tenancy	Organizations, users, roles, memberships, RBAC	Auth service; org switcher; role matrix; seed admin flow; tenant guards	5–7 days	Foundation	Domain, SSL, OIDC provider setup
Data core	BOM schema, migrations, seed data, audit tables	Postgres schema; migration framework; seed fixtures; audit ledger	5–8 days	Foundation	Managed Postgres provisioned
Agent runtime	Agent registry, model router, ACP bus, task workers	ACP contracts; worker runtime; router rules; invocation records	7–10 days	Data core	Anthropic/OpenAI/Gemini keys
Knowledge system	Ingestion, chunking, retrieval, graph, promotion	File upload; transcript import; pgvector pipeline; graph edges; approval queue	10–14 days	Agent runtime, Data core	Object storage, OCR/STT provider keys
Student OS	Student profile, progress, risk, coach interventions	Student timeline; scoring jobs; coach dashboard cards; assignment workflows	8–12 days	Knowledge system	None initially

Module	Epics and features	Representative tasks	Est. effort	Dependencies	External actions
Education OS	Courses, cohorts, lessons, transcripts, lesson summaries	Course CRUD; lesson event recording; summary generation; action items	8–12 days	Student OS, Knowledge system	Calendar integration optional
Content engine	Strategy, copy, briefs, Canva handoff, publishing readiness	Content brief workflow; asset versions; DNA validation; Canva export handoff	10–14 days	DNA engine, Knowledge system	Canva integration, brand assets
Research and competitor intel	Research packs, competitor observation, opportunity memos	Research runner; approved source lists; competitor observation records	6–10 days	Knowledge system	Public source list, compliant access rules
Communications hub	WhatsApp, Instagram, Gmail drafts and sends	Unified outbound message API; webhook ingestion; send approvals	12–18 days	Identity, Agent runtime	Meta app, webhooks, Google OAuth
Automation and approvals	Workflow editor, queueing, approvals, notifications	Temporal workflows; approval inbox; SLA reminders; retries/circuit breaker	8–12 days	Agent runtime	Email + notification providers
Analytics and digital twin	KPI warehouse, dashboards, scenario simulation	Event marts; KPI snapshots; simulation service; anomaly alerts	10–14 days	Data core, Student OS, Content engine	BI host optional
Frontend workspaces	CEO console, reviewer console, knowledge studio, ops views	App shell; dashboards; task inbox; detail pages	10–15 days	Identity, core APIs	Domain and branding

Module	Epics and features	Representative tasks	Est. effort	Dependencies	External actions
Security and compliance	Secrets, retention, traceability, policy enforcement	Secret provider adapter; retention jobs; policy middleware; audit queries	5–8 days	All core modules	Privacy policy / terms drafts
Deployment and ops	Environments, IaC, backups, monitoring, runbooks	Dev/stage/prod infra; backup validation; SLO dashboards; incident runbooks	7–10 days	Foundation onward	Cloud DNS, SSL certificates
Launch readiness	Hardening, documentation, UAT, launch checklist	E2E regression; smoke tests; App Review evidence; support docs	7–10 days	Most modules	Meta review, Google consent, Canva review if public

Representative fully-specified task cards

Example task card

Task ID: INT-WA-003
 Title: Build WhatsApp webhook ingestion and signature validation
 Priority: Critical
 Estimate: 8 hours
 Dependencies:
 - INT-WA-001 Meta app scaffolding
 - INT-WA-002 OAuth/system-user token storage

Goal:
 Receive WhatsApp webhook events, verify the handshake, persist raw events, normalize supported event types, and enqueue downstream processing.

Inputs:
 - Meta webhook verify token
 - App secret / signature verification config
 - Supported event schema list
 - Tenant mapping rules

Outputs:
 - Verified webhook endpoint

- Raw webhook_events table writes
- Normalized provider event records
- Queue messages for downstream processors

Files:

- apps/api/src/integrations/meta/webhook.controller.ts
- apps/api/src/integrations/meta/signature.ts
- apps/api/src/integrations/meta/normalizers/*.ts
- packages/contracts/src/meta/*.ts
- packages/db/prisma/migrations/...
- apps/api/test/integrations/meta/webhook.e2e.spec.ts

Database:

- webhook_endpoints
- webhook_events
- provider_jobs (optional downstream job record)

APIs required:

- Internal POST /webhooks/meta/whatsapp
- Meta webhook verification handshake

Acceptance criteria:

- Handshake succeeds with valid verify token
- Invalid signatures are rejected and logged
- Supported events persist raw payload + normalized payload reference
- Duplicate provider_event_id is idempotent
- Queue publishes downstream task on supported events

Test cases:

- Valid challenge verification
- Invalid verify token
- Invalid signature
- Duplicate event replay
- Unsupported event ignored but audited
- Queue failure creates retryable state

External actions required:

- Configure webhook callback URL in Meta App Dashboard
- Subscribe app to WhatsApp webhook fields
- Ensure public HTTPS endpoint is reachable

Definition of done:

- All acceptance criteria pass
- E2E tests pass
- Runbook entry added
- PM Mode report returned

External actions master checklist

Category	Action	Owner	When needed
Source control	Create GitHub org/repo, branch protection, environments	Human	Start
Cloud	Create AWS account or target cloud project	Human	Start
Domain	Purchase/assign domain, configure DNS	Human	Before staging
SSL	Provision certificates / managed HTTPS	Human + infra task	Before public endpoints
Database	Provision managed PostgreSQL and object storage	Human + infra task	Sprint 1
Anthropic	Create workspace, API key, set spend/rate limits	Human	Sprint 1
OpenAI	Create project/service account/API key	Human	Sprint 1
Gemini	Create AI Studio project/API key and billing if needed	Human	Sprint 1
Google Cloud	Create OAuth consent, OAuth clients, enable APIs	Human	Before Google integrations
Meta	Create Business Manager, developer app, system user, webhook config	Human	Before WhatsApp/Instagram work
Canva	Enable MFA, create integration, configure redirect URIs and scopes	Human	Before Canva work
Legal	Draft Privacy Policy, Terms, data-processing notes	Human	Before production
App reviews	Prepare screencasts, test accounts, review notes	Human + Claude docs tasks	Before launch
Monitoring	Set up logging/telemetry backends and alerting channels	Human + infra task	Before staging
Support	Shared inbox / incident channel / on-call procedure	Human	Before launch

Integration playbooks and external actions

The integration strategy should prefer **official APIs, official scopes, official review flows, and provider-native retry behavior**. That is not only the safest legal/compliance posture; it is also technically cleaner because each provider documents the required auth, permissions, and operational failure modes.

Meta playbook

For **WhatsApp Cloud API**, Meta's official getting-started flow centers on developer registration, business asset setup, creating a system user, generating a non-temporary access token, and registering a business phone number. Meta's documentation also states that `whatsapp_business_management` is required when managing WABAs not owned by your business, and that embedded signup / system-user flows are used for durable access tokens. Cloud API throughput is documented at up to 80 messages per second by default per registered number, with automatic upgrade paths as high as 1,000 mps, while messaging limits and error codes are separately enforced. ¹⁰

For **Instagram Messaging API**, Meta's official materials show that the standard baseline permissions include `instagram_basic`, `instagram_manage_messages`, and `pages_manage_metadata`, and that webhook subscriptions must be configured for messaging activity. Meta's App Review documentation states that advanced access is required before permissions can be requested from general users, and review submissions are handled through the App Review request flow. ¹¹

Meta implementation guidance for SalesOS

Item	WhatsApp	Instagram Messaging
Auth pattern	System user / business token flow	User/business authorization via Meta
Minimum permissions	<code>whatsapp_business_management</code> , <code>whatsapp_business_messaging</code>	<code>instagram_basic</code> , <code>instagram_manage_messages</code> , <code>pages_manage_metadata</code>
Critical infra	Public HTTPS webhook + verification	Public HTTPS webhook + verification
Review requirement	Needed for broader production scenarios and advanced use cases	Needed for Advanced Access / production users
Token risk	Temporary tokens expire quickly; durable token flow required	Token validity and app review state must align
Operational hazards	429s, messaging limits, template rules, phone registration	Capability mismatches, webhook subscriptions, access mode mismatch

Meta task checklist

Task	Done when
Create Business Manager and Meta app	App exists with correct use case
Create system user and assign assets	System user has required business permissions
Configure webhook callback	Verification succeeds over HTTPS
Store secrets securely	Secrets only in secret manager, never repo
Request only needed permissions	Review request minimized and evidenced

Task	Done when
Build idempotent webhook consumer	Duplicate events do not double-process
Handle <code>retry-after</code> / 429s	Backoff and circuit breaker implemented
Prepare App Review artifacts	Screencast, test accounts, step-by-step notes exist

Google playbook

Google's official OAuth 2.0 guidance for web server apps requires secure handling of confidential information, standard authorization-code exchange, access and refresh tokens, and Bearer token usage in API requests. Google also documents that OAuth clients for web apps should use HTTPS redirect URIs and valid secure origins. Drive, Gmail, and Calendar all enforce quotas, with Gmail specifically recommending truncated exponential backoff for time-based quota errors. ¹²

Google implementation guidance for SalesOS

Item	Guidance
OAuth client type	Web application for server-side auth flows
Token storage	Encrypt refresh tokens; rotate credentials; store subject and scopes metadata
Scope strategy	Start narrow and use incremental authorization where possible
Redirect URIs	HTTPS in production; exact-match configured URIs
Quota strategy	Add per-provider rate limiter, exponential backoff, monitor quotas by API
Workspace use	Domain-wide delegation only if enterprise use case actually requires it

Google integration scope suggestions for SalesOS

Use case	Likely API	Suggested base scope policy
Calendar sync	Calendar API	Read/write only if SalesOS must schedule; otherwise read-only
File ingestion	Drive API	Read-only unless exporting artifacts back to Drive
Email drafts / sends	Gmail API	Draft-first pattern, then send on approval
Docs-based knowledge	Drive + Docs export	Read-only acquisition pipeline

Canva playbook

Canva Connect's official docs state that integrations require a Canva Developer Portal integration, MFA on the developer account, and OAuth 2.0 Authorization Code + PKCE using SHA-256. The docs also note that access tokens currently expire in about four hours, and that scopes are per-user and per-resource type.

Public integrations must undergo Canva review. Canva also publishes endpoint-level rate limits, such as 20 requests/minute for design creation, 100 requests/minute for list-designs and folder listing, 60 requests/minute for autofills, and separate export throttles. Error guidance explicitly recommends exponential backoff on `too_many_requests`. ¹³

For SalesOS, Canva should be treated as a **design realization layer**, not the source of truth for content strategy. SalesOS should create approved, versioned design briefs and optionally create/edit/export designs in Canva, but canonical campaign logic should remain in SalesOS.

OpenAI playbook

OpenAI's current forward path for agentic applications is the **Responses API**, which supports stateful interactions, built-in tools, file search, web search, computer use, and function calling. OpenAI's migration guidance states that the Responses API is the future direction for building agents and that the Assistants API is being deprecated in favor of it. OpenAI also provides vector stores and retrieval APIs, with configurable chunking and expiration policies, and enforces project-specific rate limits. ¹⁴

For SalesOS, OpenAI should be used through the **Model Router** for tasks that benefit from its tool ecosystem or model performance profile. Do not couple business logic to OpenAI-specific objects; wrap provider APIs behind a provider adapter.

Anthropic playbook

Anthropic's official docs for Claude emphasize the Messages API, tool use, MCP connectivity, prompt caching, and rate-limit awareness. Prompt caching is particularly relevant for SalesOS because the constitution, DNA rules, tool definitions, and repeated large contexts are ideal cache candidates; Anthropic explicitly notes that repeated system instructions, large context documents, and tool definitions are prime use cases. Claude Code guidance further supports `CLAUDE.md`, subagents, auto mode, and verification-first workflows. ¹⁵

For SalesOS, Anthropic should be the default implementation and architecture-planning provider for Claude Code-driven development, while the runtime product itself remains provider-agnostic.

Gemini playbook

Google's Gemini API docs state that rate limits depend on usage tier, should be viewed in AI Studio, and can change over time. The docs also publish dedicated Batch API limits and note that project and key restrictions apply in AI Studio. Google's model docs distinguish stable model aliases from preview models, with previews often carrying tighter limits and shorter lifetimes. ¹⁶

For SalesOS, Gemini is best treated as a **secondary provider** in the router for selected workloads, especially where price/latency tradeoffs are attractive. For production routing, prefer stable model names, not preview aliases, unless a clear benchmark win justifies the churn risk. ¹⁷

Tech stack, roadmap, and deliverables

Recommended tech stack

The recommended stack below prioritizes durability, vendor neutrality, API-first design, and Claude Code friendliness.

Layer	Recommendation	Rationale
Monorepo	<code>pnpm</code> + Turborepo	<code>pnpm</code> natively supports workspaces; Turborepo is built for scaling multi-package task execution in monorepos. ¹⁸
Frontend	Next.js + React + TypeScript	Strong full-stack ergonomics, good component ecosystem, easy dashboarding
Backend API	NestJS + TypeScript	Explicit module boundaries, DTOs, guards, testability, suitable for large API-first systems
Background orchestration	Temporal	Durable execution, retries, scalable workflow semantics for long-running approvals/integration jobs. ¹⁹
Database	PostgreSQL	Strong relational model, <code>jsonb</code> , mature transactional features, row-level security. ²⁰
Vector retrieval	pgvector on PostgreSQL	Keeps RAG inside the relational core; HNSW available for performance. ²¹
Object storage	S3-compatible storage	Vendor-portable, cheap, good for recordings, exports, raw uploads
Cache / ephemeral queueing	Redis	Rate limits, dedupe windows, short-lived coordination
Auth	OIDC/SAML-capable IdP; default self-hostable option is Keycloak, managed option acceptable if speed matters	Multi-tenant SaaS and enterprise readiness
Search / graph access	Start with Postgres + pgvector + edge tables; add graph service only if proven necessary	Keeps v1 simpler while preserving graph semantics
Observability	OpenTelemetry + Collector + Grafana/ Prometheus/Loki + Sentry	Vendor-neutral telemetry plus app error tracking. ²²

Layer	Recommendation	Rationale
CI/CD	GitHub Actions + Docker + IaC	Good Claude Code compatibility and review ergonomics
Infra	AWS default deployment target: ECS/ Fargate or EKS later if needed; RDS/ Postgres	Sensible production baseline without overcommitting to Kubernetes on day one
Secrets	Cloud secret manager + runtime injection	No <code>.env</code> secrets in repo or CI logs
LLM provider layer	Provider adapters for Anthropic, OpenAI, Gemini	Preserves vendor choice and fallback capability
STT / OCR	Pluggable adapters	Avoid lock-in; let model router or provider config select best option

Prioritized backlog

Rank	ID	Backlog item	Business impact	Est. effort	Dependencies
1	FND-001	Monorepo, CI, coding standards, CLAUDE package	Foundational	5d	None
2	DAT-001	Core Postgres schema for BOM + audit	Foundational	7d	FND-001
3	IAM-001	Organizations, users, roles, memberships, tenant guards	Foundational	6d	DAT-001
4	AGT-001	ACP contracts + agent registry + invocation logging	Critical differentiator	8d	DAT-001
5	GOV-001	Approval engine + audit trail + rollback patterns	Critical differentiator	6d	IAM-001, AGT-001
6	KNO-001	File ingestion + knowledge item/version pipeline	Critical differentiator	8d	AGT-001
7	KNO-002	pgvector retrieval + graph edge relationships	Critical differentiator	6d	KNO-001
8	DNA-001	Organizational DNA Engine + brand rules evaluation	High	4d	KNO-001
9	EDU-001	Course / lesson / transcript pipeline	High	7d	DAT-001
10	STD-001	Student timeline + risk profiling + coach actions	High	8d	EDU-001, KNO-002

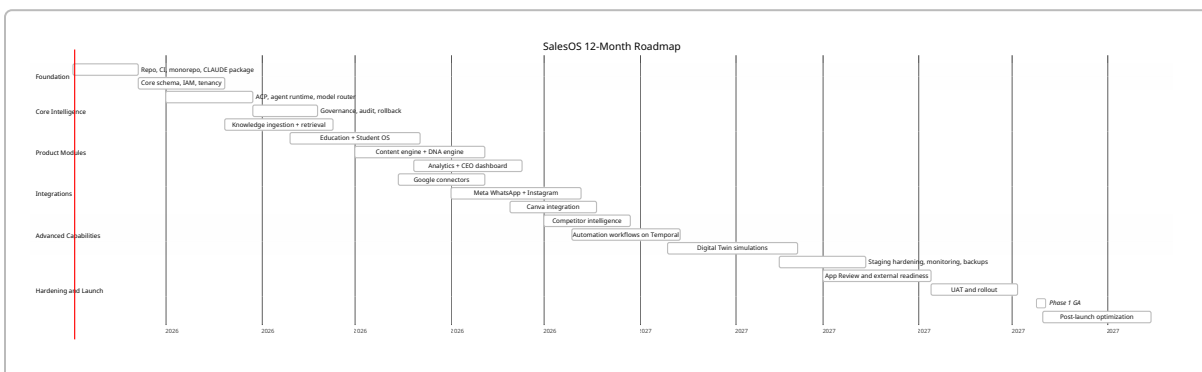
Rank	ID	Backlog item	Business impact	Est. effort	Dependencies
11	CNT-001	Content brief → draft → QA → approval workflow	High	8d	DNA-001, GOV-001
12	INT-META-001	Meta app, WhatsApp webhooks, token storage	High	10d	GOV-001, IAM-001
13	INT-GGL-001	Google OAuth + Drive/Gmail/Calendar connectors	High	8d	IAM-001
14	INT-CANVA-001	Canva integration + design brief/export path	High	8d	CNT-001
15	ANA-001	KPI snapshots + CEO dashboard	High	7d	DAT-001
16	SIM-001	Digital Twin scenario service	Medium-high	8d	ANA-001, STD-001
17	CMP-001	Competitor observation pipeline	Medium-high	6d	KNO-002
18	AUT-001	Workflow runtime on Temporal + notifications	High	8d	AGT-001
19	OPS-001	Staging/prod, monitoring, backups, runbooks	High	7d	FND-001 onward
20	LCH-001	Launch hardening, App Review evidence, UAT	Critical before release	10d	Most above

Ninety-day sprint plan

Sprint	Focus	Exit criteria
Sprint Alpha	Repo, infra baseline, CLAUDE package, data core	Monorepo live; CI green; base schema migrated
Sprint Bravo	Identity, tenancy, agent runtime, ACP	User/org model works; agents invoke via typed contracts
Sprint Charlie	Governance, audit, knowledge ingestion	Approval queue live; auditable knowledge pipeline works
Sprint Delta	Education + Student OS	Lessons/transcripts produce summaries and student updates
Sprint Echo	Content engine + DNA engine + Canva stub	Content workflow complete through approval

Sprint	Focus	Exit criteria
Sprint Foxtrot	Meta + Google integrations + CEO dashboard	Real webhook/event flow in staging; operational dashboard live

Twelve-month roadmap



Rollout strategy

The recommended rollout path is:

Phase	Audience	Enabled modules	Objective
Dev internal	Founder only	Knowledge, content, approvals, analytics	Stabilize core architecture
Staging internal	Founder + 1-2 operators	Student OS, lesson pipeline, connectors in sandbox	Validate workflows
Controlled beta	Selected real S.O.S. users	WhatsApp/Google/Canva subset	Observe operational load and approval burden
Phase 1 GA	Internal S.O.S. production	Core modules + safe integrations	Replace fragmented tools for core operations
Phase 2 productization	Multi-tenant pilot	Tenant controls, package overlays	Prepare commercial version

Launch checklist

Area	Checklist
Product	Core workflows tested end-to-end; all P0 and P1 bugs closed
AI	Model router active; prompt package versioned; fallback providers configured
Governance	Approval rules verified; audit queries tested; rollback playbooks written

Area	Checklist
Integrations	All tokens live; webhooks verified; App Review / consent screens approved
Security	Secrets externalized; RBAC verified; tenant isolation tested; logs sanitized
Reliability	Backups tested; restore tested; alerts configured; error budgets defined
Legal	Privacy Policy, Terms, consent notices, retention policy published
Support	Incident channel, support inbox, on-call owner, launch-day runbook ready
Analytics	KPI dashboard live; usage/cost telemetry live
Documentation	Product Bible, Engineering Bible, Ops runbooks, admin docs exported

Final deliverables

The final package should be exported as **Markdown, Docx, and PDF**, with the repository preserving the source-of-truth Markdown files. Anthropic's documentation reinforces the value of git-tracked `CLAUDE.md`, persistent instructions, and subagent files; that aligns directly with a doc-first repository structure. ²³

Recommended folder structure:

```
salesos/
├─ CLAUDE.md
├─ README.md
├─ apps/
│   ├─ web/
│   ├─ api/
│   └─ worker/
├─ packages/
│   ├─ contracts/
│   ├─ db/
│   ├─ ui/
│   ├─ config/
│   ├─ observability/
│   └─ ai/
├─ .claude/
│   ├─ agents/
│   │   ├─ security-reviewer.md
│   │   ├─ qa-reviewer.md
│   │   └─ integration-specialist.md
│   └─ data-architect.md
├─ settings.json
├─ hooks/
├─ docs/
│   └─ 00-constitution/
```

```
| | └─ constitution.md
| └─ 01-vision/
| | └─ executive-summary.md
| | └─ company-model.md
| └─ 02-architecture/
| | └─ master-blueprint.md
| | └─ agent-catalog.md
| | └─ acp-spec.md
| └─ 03-data/
| | └─ bom-and-erd.md
| | └─ migrations-policy.md
| └─ 04-governance/
| | └─ ai-governance.md
| | └─ approval-levels.md
| | └─ rollback-and-versioning.md
| └─ 05-integrations/
| | └─ meta-whatsapp-instagram.md
| | └─ google.md
| | └─ canva.md
| | └─ openai.md
| | └─ anthropic.md
| | └─ gemini.md
| └─ 10-roadmap/
| | └─ 12-month-roadmap.md
| | └─ 90-day-sprint-plan.md
| └─ 20-execution/
| | └─ current-sprint.md
| | └─ backlog.md
| | └─ task-cards/
| └─ 30-ops/
| | └─ deployment.md
| | └─ monitoring.md
| | └─ backups.md
| | └─ incident-response.md
| └─ adr/
└─ prompts/
  └─ root-system-prompt.md
  └─ task-prompt-template.md
  └─ pm-mode-template.md
  └─ integration-review-template.md
└─ exports/
  └─ Product-Bible.pdf
  └─ Product-Bible.docx
  └─ Product-Bible.md
```

This structure is the right “single prompt package” in practical terms: one canonical body of instructions, decomposed into files that Claude Code can reliably load, reason over, and execute against without drowning in context.

-
- 1 4 9 23 **Best practices for Claude Code - Claude Code Docs**
<https://code.claude.com/docs/en/best-practices>
 - 2 19 **<https://docs.temporal.io/evaluate/understanding-temporal>**
<https://docs.temporal.io/evaluate/understanding-temporal>
 - 3 **Permissions - Meta for Developers - Facebook**
https://developers.facebook.com/documentation/business-messaging/whatsapp/permissions/?utm_source=chatgpt.com
 - 5 8 **Effective context engineering for AI agents \ Anthropic**
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
 - 6 22 **<https://opentelemetry.io/docs/>**
<https://opentelemetry.io/docs/>
 - 7 20 **<https://www.postgresql.org/docs/current/datatype-json.html>**
<https://www.postgresql.org/docs/current/datatype-json.html>
 - 10 **WhatsApp Cloud API Get Started - Meta for Developers**
https://developers.facebook.com/documentation/business-messaging/whatsapp/get-started?utm_source=chatgpt.com
 - 11 **Getting Started - Meta for Developers - Facebook**
https://developers.facebook.com/documentation/business-messaging/instagram-messaging/get-started?utm_source=chatgpt.com
 - 12 **<https://developers.google.com/identity/protocols/oauth2/web-server>**
<https://developers.google.com/identity/protocols/oauth2/web-server>
 - 13 **<https://www.canva.dev/docs/connect/creating-integrations/>**
<https://www.canva.dev/docs/connect/creating-integrations/>
 - 14 **<https://developers.openai.com/api/reference/responses/overview/>**
<https://developers.openai.com/api/reference/responses/overview/>
 - 15 **<https://platform.claude.com/docs/en/build-with-claude/working-with-messages>**
<https://platform.claude.com/docs/en/build-with-claude/working-with-messages>
 - 16 **<https://ai.google.dev/gemini-api/docs/rate-limits>**
<https://ai.google.dev/gemini-api/docs/rate-limits>
 - 17 **Models | Gemini API - Google AI for Developers**
https://ai.google.dev/gemini-api/docs/models?utm_source=chatgpt.com
 - 18 **<https://pnpm.io/workspaces>**
<https://pnpm.io/workspaces>
 - 21 **<https://github.com/pgvector/pgvector>**
<https://github.com/pgvector/pgvector>